

Rapport de Projet - Application Microservices

Containerisation & Orchestration avec Docker / Kubernetes

Valentin-Even NGUYEN

1. Introduction

EventHub est une application web de gestion d'événements et de participants, initialement développée sous forme d'une application monolithique avec un backend Django REST Framework et un frontend React. Dans le cadre de ce projet, l'application a été décomposée en microservices indépendants, containerisés avec Docker et orchestrés avec Kubernetes, avec intégration d'Istio comme service mesh.

2. Architecture Globale

L'application est composée de quatre composants principaux déployés dans un cluster Kubernetes Minikube :

Frontend (React)	Interface utilisateur SPA	Port 3000
auth-service (Django)	Gestion utilisateurs et tokens JWT	Port 8001
event-service (Django)	Gestion événements, participants, inscriptions	Port 8000
PostgreSQL	Base de données relationnelle (2 bases)	Port 5432

Le flux de communication suit le schéma suivant :



```
val@val:~/projet-prog-dist$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
auth-service-684b664d59-n8t1m      2/2     Running   1 (38s ago) 47s
event-service-7768cfccbf-jggfp     2/2     Running   1 (39s ago) 47s
frontend-service-6c848cb677-zkb12  2/2     Running   0           47s
postgres-778779d597-72tcz          2/2     Running   0           47s
```

```
val@val:~/projet-prog-dist$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
auth-service	ClusterIP	10.109.84.171	<none>	80/TCP	3d22h
event-service	ClusterIP	10.98.78.26	<none>	80/TCP	3d22h
frontend-service	NodePort	10.97.77.57	<none>	80:31754/TCP	3d22h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	3d22h
postgres	NodePort	10.101.126.164	<none>	5432:32533/TCP	3d22h


```
val@val:~/projet-prog-dist$ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
auth-service	1/1	1	1	3d20h
event-service	1/1	1	1	3d21h
frontend-service	1/1	1	1	3d22h
postgres	1/1	1	1	3d22h

3. Decomposition en Microservices

3.1 auth-service

Ce service gere exclusivement l'authentification et les utilisateurs. Il expose trois endpoints :

- POST /api/auth/login/ : authentification, retourne un token + role
- POST /api/auth/register/ : creation de compte avec role viewer par default

Les roles sont de deux types : admin (acces complet au CRUD) et viewer (lecture seule).

3.2 event-service

Ce service gere l'ensemble de la logique metier de l'application :

- Modele Event : titre, description, date, statut (draft/active/completed/cancelled)
- Modele Participant : nom, email unique
- Modele Registration : relation many-to-many entre Event et Participant, avec contrainte unique_together

Pour chaque requete, event-service appelle auth-service via HTTP pour valider le token et recuperer le role de l'utilisateur avant d'autoriser l'operation.

3.3 Frontend React

L'interface utilisateur est une SPA React. L'authentification est geree via localStorage et un intercepteur Axios injecte automatiquement le token dans chaque requete.

EventHub

Dashboard

Événements

Participants

Déconnexion

Tableau de bord

Bienvenue sur EventHub

0

ÉVÉNEMENTS

0

PARTICIPANTS

0

INSCRIPTIONS

Événements récents

Voir tout →

Aucun événement pour le moment.

EventHub

Dashboard

Événements

Participants

Déconnexion

Tous les statuts

jj/mm/aaaa

Créer

ACTIF

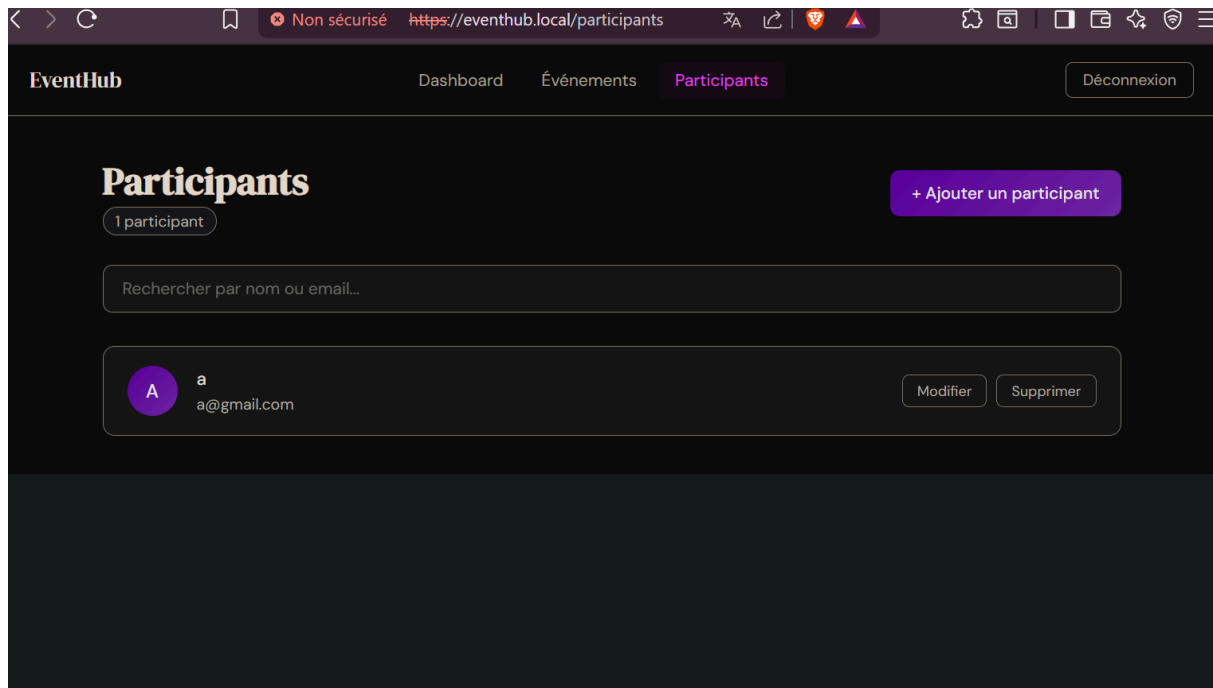
26 avr. 2026

e

e

Modifier

Supprimer



4. Containerisation Docker

Chaque microservice possède son propre Dockerfile. Exemple pour event-service :

```
FROM python:3.13

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["sh", "-c", "python manage.py migrate && python manage.py\nrunserver 0.0.0.0:8000"]
```

5. Orchestration Kubernetes

5.1 Deployments

Chaque composant est déployé avec un Deployment Kubernetes définissant le nombre de replicas, l'image Docker, les ports et les variables d'environnement. Les variables sensibles (credentials de base de données, URL des services) sont injectées via des champs env dans les fichiers de déploiement.

5.2 Services

Service	Type	Port	Role
auth-service	ClusterIP	8001	Communication interne
event-service	ClusterIP	8000	Communication interne
frontend-service	NodePort	3000	Acces via Ingress
postgres	ClusterIP	5432	Base de données interne

5.3 Ingress NGINX

Un Ingress NGINX expose l'application via un point d'entrée unique sur eventhub.local avec le routage suivant :

- /api/auth → auth-service:8001
- /api/events → event-service:8000
- / → frontend-service:3000

L'accès depuis la machine locale est configuré via /etc/hosts en mappant l'IP Minikube au domaine eventhub.local.

```
val@val:~/projet-prog-dist$ kubectl get ingress
NAME                CLASS    HOSTS          ADDRESS          PORTS    AGE
eventhub-ingress    nginx    eventhub.local 192.168.49.2    80       3d22h
```

5.4 RBAC Kubernetes

Un Role et un RoleBinding ont été configurés pour sécuriser l'accès aux ressources du cluster. Le ServiceAccount default dispose des permissions de lecture sur les pods :

```
Kind: Role / RoleBinding
Resources: pods
Verbs: get, list
```

6. Base de Données PostgreSQL

Deux bases de données séparées sont utilisées, une par service, respectant le principe de séparation des données dans une architecture microservices :

- authdb : tables Django natives (auth_user, authtoken_token, auth_group)
- eventdb : tables métier (api_event, api_participant, api_registration)

La base de données est initialisée via un ConfigMap Kubernetes montant un script SQL dans /docker-entrypoint-initdb.d. Les credentials sont passées via des variables d'environnement dans les deployments.

Note : le volume de données utilise emptyDir pour simplifier le déploiement en environnement de développement. Les données ne sont pas persistées entre les redémarrages du pod, ce qui est acceptable dans le cadre de ce projet.

7. Service Mesh avec Istio

Istio a été intégré au cluster Minikube avec injection automatique des sidecars (istio-proxy) dans chaque pod. Cela apporte les bénéfices suivants :

- mTLS automatique entre tous les services (chiffrement des communications internes)
- Observabilité : métriques Prometheus exposées sur chaque pod
- Gestion avancée du trafic via les VirtualService et DestinationRule

Probleme rencontre : l'Ingress NGINX communiquait en HTTP simple alors qu'Istio attendait du mTLS, provoquant des erreurs 502. La solution a ete de configurer Istio en mode PERMISSIVE, permettant d'accepter a la fois HTTP et mTLS :

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
spec:
  mTLS:
    mode: PERMISSIVE
```

8. Securite

Plusieurs couches de securite ont ete mises en place :

- RBAC Kubernetes : controle d'accès aux ressources du cluster
- mTLS Istio : chiffrement des communications entre services
- HTTPS : certificat TLS auto-signé configuré sur l'Ingress

9. Difficultes Rencontrees

Volume PostgreSQL corrompu	Migration vers emptyDir
Race condition Django / Postgres	Script d'attente netcat dans le Dockerfile
Erreur 502 avec Istio	Mode PERMISSIVE sur PeerAuthentication
Version PostgreSQL incompatible	Mise a jour de postgres:10.1 vers postgres:15
Variables d'environnement absentes	Correction des fichiers de deployment

10. Conclusion

Le projet EventHub illustre une migration complete d'une application monolithique vers une architecture microservices deployee sur Kubernetes. Les technologies mises en oeuvre couvrent l'ensemble du programme : containerisation Docker, orchestration Kubernetes (Deployments, Services, Ingress, ConfigMaps, RBAC), service mesh Istio et base de donnees PostgreSQL.

Le code source est disponible sur GitHub.